{EPITECH}

RTC_

< BOOTSTRAP - RUST & AXUM />

RTC

Discover Rust?

Before diving into code, take 10 minutes to explore and answer these questions:

Research Questions

About rust:

- ✓ When was Rust created and by whom?
- ✓ What problem was Rust designed to solve?
- ✓ Name 3 major companies or projects using Rust in production

Core Concepts:

- ✓ What is ”memory safety without garbage collection”?
- ✓ What makes Rust different from C/C++?
- ✓ What makes Rust different from Python/JavaScript?

The Ownership System:

- ✓ What is ”ownership” in Rust?
- ✓ What are the three ownership rules?
- ✓ Why does Rust not have null pointers?

Practical Use:

- ✓ Why would you choose Rust for a web server?
- ✓ What is the ”zero-cost abstraction” principle?
- ✓ What does ”fearless concurrency” mean?

Fun fact: Discord switched critical services from Go to Rust and saw latency improvements and reduced CPU usage.

Goal: Understand why Rust exists before learning how to use it.

Rust tasks (2h)

Basics

Task 1.1 - Functions and types

Create these functions:

- ✓ `double(n: i32)-> i32` - Multiplies by 2
- ✓ `is_adult(age: u8)-> bool` - Checks if >= 18
- ✓ `greet(name: String)-> String` - Returns "Hello, {name}!"

Task 1.2 - Mutability

Create a function `increment_all(mut numbers: Vec<i32>)-> Vec<i32>` that adds 1 to each number.

References & Borrowing

Task 2.1 - String vs &str

Create these functions:

- ✓ `get_first_word(text: &str)-> &str` - Returns the first word
- ✓ `make_uppercase(text: &str)-> String` - Returns uppercase version
- ✓ `count_chars(text: &str)-> usize` - Counts characters

Task 2.2 - Mutable references

Create a function `append_suffix(text: &mut String, suffix: &str)` that modifies the string in place.

Task 2.3 - Vec with references

On a `Vec<String>`:

- ✓ `find_by_prefix(words: &Vec<String>, prefix: &str)-> Option<&String>`
- ✓ `total_length(words: &Vec<String>)-> usize` - Returns the sum of each String

{EPITECH}

Option/Result

Task 3.1 - Option

Create these functions:

- ✓ `find_max(numbers: Vec<i32>)-> Option<i32>` - Largest number (None if empty)
- ✓ `get_at_index(vec: Vec<String>, index: usize)-> Option<String>`

Task 3.2 - Result

Create these functions that return `Result<T, String>`:

- ✓ `divide(a: f64, b: f64)-> Result<f64, String>` - Error if division by zero
- ✓ `parse_age(input: &str)-> Result<u8, String>` - Parse and validate 0-120

Task 3.3 - Chaining

Create a function `parse_and_double(input: &str)-> Result<i32, String>` that:

- ✓ Parses a number
- ✓ Multiplies it by 2
- ✓ Uses `?` to propagate errors

Struct & Collections

Task 4.1 - Struct

Create `Person` struct:

```
struct Person {
    name: String,
    age: u8,
    email: String,
}
```

Implement:

- ✓ `new(name: String, age: u8, email: String)-> Self`
- ✓ `is_adult(&self)-> bool`
- ✓ `display(&self)-> String` - Format: "Name (age) - email"

Task 4.2 - HashMap

Create a phone directory:

- ✓ `HashMap<String, String>` (name -> phone)
- ✓ Function `add_contact(map: &mut HashMap<String, String>, name: String, phone: String)`
- ✓ Function `remove_contact(map: &mut HashMap<String, String>, name: &str)-> bool`

Modules & Derives

Task 5.1 - Organization

Create three files in `src/`:

- ✓ `models.rs`
- ✓ `utils.rs`
- ✓ `main.rs`

In `models.rs`:

```
#[derive(Debug, Clone)]
pub struct User {
    pub id: u32,
    pub username: String,
}
```

In `utils.rs`:

```
pub fn is_valid_username(username: &str) -> bool {
    username.len() >= 3 && username.len() <= 20
}
```

In `main.rs`: import and use them.

Axum - First Endpoints (1h)

Setup

Add to `Cargo.toml`:

```toml
[dependencies]
axum = "0.8"
tokio = { version = "1", features = ["full"] }
serde = { version = "1", features = ["derive"] }
serde_json = "1"
uuid = { version = "1", features = ["v4", "serde"] }
```

Endpoint 1 - Basic Router

Create a simple server with health check:

```rust
use axum::{routing::get, Router, Json};
use serde::Serialize;

#[derive(Serialize)]
struct HealthResponse {
    status: String,
}

async fn health() -> Json<HealthResponse> {
    // To implement
}

#[tokio::main]
async fn main() {
    let app = Router::new()
        .route("/health", get(health));

    let listener = tokio::net::TcpListener::bind("0.0.0.0:3000")
        .await
        .unwrap();

    axum::serve(listener, app).await.unwrap();
}
```

Test it: `curl http://localhost:3000/health`

Endpoint 2 - Server Model

Define the Server structure:

```
use uuid::Uuid;

#[derive(Clone, Serialize, Deserialize)]
struct Server {
    // Time to think which informations you need
}

#[derive(Deserialize)]
struct CreateServerPayload {
    // To implement
}
```

Create an in-memory storage:

```
use std::sync::Arc;
use tokio::sync::Mutex;
use std::collections::HashMap;

type SharedServers = Arc<Mutex<HashMap<Uuid, Server>>>;

#[derive(Clone)]
struct AppState {
    servers: SharedServers,
}
```

Obviously this is an in-memory storage so you will have to change it later with your database

Endpoint 3 - Create Server

Implement `POST /servers`:

```
use axum::{extract::State, routing::post};

async fn create_server (
    State(state): State<AppState>,
    Json(payload): Json<CreateServerPayload>,
) -> Json<Server> {
    let server = Server {
        // To implement
    }
```

```
    let mut servers = state.servers.lock().await;
    // Insert your data into the in-memory storage

    Json(server)
}
```

You need to add this new route to your router

> Don't forget to include the `AppState`

Test it:

```
curl -X POST http://localhost:3000/servers \
    -H "Content-Type: application/json" \
    -d '{"name":"Gaming Server","description":"For gamers"}'
```

Endpoint 4 - List Servers

Implement `GET /servers`:

```
async fn list_servers(
    State(state): State<AppState>,
) -> Json<Vec<Server>> {
    let servers = // To implement
    let server_list: Vec<Server> = // To implement
    Json(server_list)
}
```

> Don't forget to add it to your router

Test it: `curl http://localhost:3000/servers`

Endpoint 5 - Get Server by ID

Implement `GET /servers/{id}`:

```rust
use axum::extract::Path;
use axum::http::StatusCode;

async fn get_server(
    State(state): State<AppState>,
    Path(id): Path<Uuid>,
) -> Result<Json<Server>, StatusCode> {
    let servers = state.servers.lock().await;

    servers.get() // You need to find what to get
        .cloned()
        .map(Json)
        .ok_or(StatusCode::NOT_FOUND)
}
```

Test it: `curl http://localhost:3000/servers/{SERVER_ID}`

Endpoint 6 - Update & Delete

I hope you understand how does it works so now implement these 2 routes:

- ✓ `PUT /servers/{id}`
- ✓ `DELETE /servers/{id}`

Test it:

```bash
# Update
curl -X PUT http://localhost:3000/servers/{SERVER_ID} \
    -H "Content-Type: application/json" \
    -d '{"name":"Updated Name"}'

# Delete
curl -X DELETE http://localhost:3000/servers/{SERVER_ID}
```

v 1.1.1

{EPITECH}